

Yarfi Setya Nugraha - 140810240003
Praktikum Kriptografi

Soal1: Enkripsikan **MAGANG** dengan $K = \begin{bmatrix} 7 & 6 \\ 2 & 5 \end{bmatrix}$ Tuliskan setiap langkah-langkahnya!

Jawab:

$P = \text{MAGANG}$

$K = \begin{bmatrix} 7 & 6 \\ 2 & 5 \end{bmatrix}$

Karena kunci berukuran 2×2 , plaintext dibagi menjadi blok yang masing-masing berisi 2 huruf:

$\text{MAGANG} \rightarrow \text{MA} \mid \text{GA} \mid \text{NG}$

Dengan aturan $A = 0, B = 1, \dots, Z = 25$:

$M = 12, A = 0, G = 6, N = 13$

Maka masing-masing blok menjadi:

Huruf	Nilai
M	12
A	0
G	6
A	0
N	13
G	6

Rumus enkripsi Hill Cipher:

$$C = K \times P \pmod{26}$$

Blok pertama: MA

Blok	P1	P2	$K_{11} \cdot P_1 + K_{12} \cdot P_2$	$K_{21} \cdot P_1 + K_{22} \cdot P_2$	C1 mod 26	C2 mod 26	Ciphertext
MA	12	0	84	24	6	24	GY

Jadi diperoleh 6,246,24 atau **GW**.

Blok kedua: GA

GA	6	0	42	12	16	12	QM
----	---	---	----	----	----	----	----

Jadi diperoleh 16,1216,12 atau **QM**.

Blok ketiga: NG

NG	13	6	127	56	23	4	XE
----	----	---	-----	----	----	---	----

Jadi diperoleh 23,423,4 atau **XE**.

Dengan demikian seluruh hasil enkripsi:

GW + QM + XE

Hasil Akhir	GYQMXE
--------------------	---------------

Soal 2: Enkripsikan nama lengkap anda menggunakan Affine Cipher dan kembalikan menjadi plainteks, **a=9 b=[2 digit NPM akhir]**.

Jawab:

Nama: YARFI SETYA NUGRAHA

2 digit terakhir NPM: 03

Jadi a = 9

b = 3 (nilai numeriknya 3, bukan 03)

Rumus yang dipakai:

$$E(x) = (ax + b) \bmod 26$$

$$E(x) = (9x + 3) \bmod 26$$

Untuk dekripsi

$$D(y) = a^{-1} (y - b) \bmod 26$$

Pertama kita mencari invers modulo dari 9 terhadap 26.

$$9^{-1} \bmod 26 = 3$$

Konversi nama ke angka:

Y	24
A	0
R	17
F	5
I	8
S	18
E	4
T	19
Y	24
A	0
N	13
U	20
G	6
R	17
A	0
H	7
A	0

Cara enkripsi:

$9x + 3$	$y = \text{mod } 26$
219	11
3	3
156	0
48	22
75	23
165	9
39	13
174	18
219	11
3	3
120	16
183	1
57	5
156	0
3	3
66	14
3	3

Cipher teks yang terbentuk:

Ciphertext
L
D
A
W
X
J
N
S
L
D
Q
B
F
A
D
O
D

Dekripsi:

y	$3(y - 3)$	$x = \text{mod } 26$	Dekripsi
11	24	24	Y
3	0	0	A
0	-9	17	R
22	57	5	F
23	60	8	I
9	18	18	S
13	30	4	E
18	45	19	T
11	24	24	Y
3	0	0	A
16	39	13	N
1	-6	20	U
5	6	6	G
0	-9	17	R
3	0	0	A
14	33	7	H
3	0	0	A

No	Plaintext	x	$9x + 3$	$y = \text{mod } 26$	Ciphertext	y	$3(y - 3)$	$x = \text{mod } 26$	Dekripsi
1	Y	24	219	11	L	11	24	24	Y
2	A	0	3	3	D	3	0	0	A
3	R	17	156	0	A	0	-9	17	R
4	F	5	48	22	W	22	57	5	F
5	I	8	75	23	X	23	60	8	I
6	S	18	165	9	J	9	18	18	S
7	E	4	39	13	N	13	30	4	E
8	T	19	174	18	S	18	45	19	T
9	Y	24	219	11	L	11	24	24	Y
10	A	0	3	3	D	3	0	0	A
11	N	13	120	16	Q	16	39	13	N
12	U	20	183	1	B	1	-6	20	U
13	G	6	57	5	F	5	6	6	G
14	R	17	156	0	A	0	-9	17	R
15	A	0	3	3	D	3	0	0	A
16	H	7	66	14	O	14	33	7	H
17	A	0	3	3	D	3	0	0	A

Soal3: Buatlah program untuk enkripsi, dekripsi, dan mencari kunci Hill Cipher (bahasa pemrograman bebas)

Jawab:

```
import math

MODULUS = 26

def text_to_numbers(text):
    text = text.upper().replace(" ", "")

    if not text.isalpha() or not text.isascii():
        raise ValueError("Teks hanya boleh berisi huruf A-Z.")

    return [ord(char) - ord("A") for char in text]

def numbers_to_text(numbers):
    return "".join(chr((number % MODULUS) + ord("A")) for number in numbers)

def input_key():
    order = int(input("Masukkan ordo matriks key: "))

    if order < 2:
        raise ValueError("Ordo matriks minimal 2.")

    key = []

    print(f"Masukkan key {order} x {order}:")

    for i in range(order):
        row = list(map(int, input(f"Baris {i + 1}: ").split()))

        if len(row) != order:
            raise ValueError(
                f"Setiap baris harus memiliki {order} angka."
```

```

    )

    key.append([value % MODULUS for value in row])

    return key

def determinant(matrix):
    size = len(matrix)

    if size == 1:
        return matrix[0][0]

    if size == 2:
        return (
            matrix[0][0] * matrix[1][1]
            - matrix[0][1] * matrix[1][0]
        )

    result = 0

    for column in range(size):
        minor = [
            [
                matrix[row][item]
                for item in range(size)
                if item != column
            ]
            for row in range(1, size)
        ]

        sign = 1 if column % 2 == 0 else -1

        result += (
            sign
            * matrix[0][column]
            * determinant(minor)
        )

    return result

```

```

def modular_inverse(number):
    number %= MODULUS

    for candidate in range(MODULUS):
        if (number * candidate) % MODULUS == 1:
            return candidate

    raise ValueError("Invers modulo tidak ditemukan.")

def validate_key(key):
    det = determinant(key) % MODULUS

    if math.gcd(det, MODULUS) != 1:
        raise ValueError(
            "Key tidak valid karena determinannya "
            "tidak relatif prima dengan 26."
        )

def inverse_key(key):
    validate_key(key)

    size = len(key)

    # Untuk matriks 2x2
    if size == 2:
        a = key[0][0]
        b = key[0][1]
        c = key[1][0]
        d = key[1][1]

        det = (a * d - b * c) % MODULUS
        det_inverse = modular_inverse(det)

        inverse = [
            [
                d * det_inverse % MODULUS,

```

```

        -b * det_inverse % MODULUS
    ],
    [
        -c * det_inverse % MODULUS,
        a * det_inverse % MODULUS
    ]
]

return inverse

raise ValueError(
    "Pencarian invers saat ini digunakan untuk key 2x2."
)

def multiply_matrix_vector(matrix, vector):
    size = len(matrix)

    result = []

    for row in range(size):
        total = 0

        for column in range(size):
            total += matrix[row][column] * vector[column]

        result.append(total % MODULUS)

    return result

def process_text(text, key):
    numbers = text_to_numbers(text)

    block_size = len(key)

    # Tambahkan X jika jumlah huruf tidak habis dibagi ukuran key
    while len(numbers) % block_size != 0:
        numbers.append(ord("X") - ord("A"))

```

```

result = []

for index in range(0, len(numbers), block_size):
    block = numbers[index:index + block_size]

    encrypted_block = multiply_matrix_vector(
        key,
        block
    )

    result.extend(encrypted_block)

return numbers_to_text(result)

def encryption():
    plain_text = input("Masukkan plaintext: ")

    key = input_key()

    validate_key(key)

    cipher_text = process_text(plain_text, key)

    print("CIPHERTEXT:", cipher_text)

def decryption():
    cipher_text = input("Masukkan ciphertext: ")

    key = input_key()

    inverse = inverse_key(key)

    plain_text = process_text(cipher_text, inverse)

    print("PLAINTEXT:", plain_text)

def multiply_matrix(left, right):

```

```

size = len(left)

result = []

for row in range(size):
    new_row = []

    for column in range(size):
        total = 0

        for item in range(size):
            total += (
                left[row][item]
                * right[item][column]
            )

        new_row.append(total % MODULUS)

    result.append(new_row)

return result

def find_key():
    order = int(input("Masukkan ordo matriks key: "))

    if order != 2:
        raise ValueError(
            "Fitur mencari key dibuat untuk matriks 2x2."
        )

    plain_text = input(
        "Masukkan 4 huruf plaintext yang diketahui: "
    )

    cipher_text = input(
        "Masukkan 4 huruf ciphertext pasangannya: "
    )

    plain_numbers = text_to_numbers(plain_text)

```

```

cipher_numbers = text_to_numbers(cipher_text)

if len(plain_numbers) != 4 or len(cipher_numbers) != 4:
    raise ValueError(
        "Plaintext dan ciphertext harus tepat 4 huruf."
    )

# Membentuk matriks plaintext
plain_matrix = [
    [plain_numbers[0], plain_numbers[2]],
    [plain_numbers[1], plain_numbers[3]]
]

# Membentuk matriks ciphertext
cipher_matrix = [
    [cipher_numbers[0], cipher_numbers[2]],
    [cipher_numbers[1], cipher_numbers[3]]
]

plain_inverse = inverse_key(plain_matrix)

key = multiply_matrix(
    cipher_matrix,
    plain_inverse
)

print("KEY:")

for row in key:
    print(*row)

def menu():
    print()
    print("==== HILL CIPHER =====")
    print("1. Enkripsi")
    print("2. Dekripsi")
    print("3. Cari Kunci")
    print("4. Keluar")

```

```
        return input("Masukkan pilihan: ")

def main():
    while True:
        pilihan = menu()

        try:
            if pilihan == "1":
                encryption()

            elif pilihan == "2":
                decryption()

            elif pilihan == "3":
                find_key()

            elif pilihan == "4":
                print("Program selesai.")
                break

            else:
                print("Pilihan tidak valid.")

        except ValueError as error:
            print("Error:", error)

if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\Yarfi Setya Nugraha\OneDrive\Desktop\VSCode project\Kriptografi\tugas2> py hillcipher.py
```

```
===== HILL CIPHER =====
```

1. Enkripsi
2. Dekripsi
3. Cari Kunci
4. Keluar

Masukkan pilihan: 1

Masukkan plaintext: magang

Masukkan ordo matriks key: 2

Masukkan key 2 x 2:

Baris 1: 7 6

Baris 2: 2 5

CIPHERTEXT: GYQMXE

```
===== HILL CIPHER =====
```

1. Enkripsi
2. Dekripsi
3. Cari Kunci
4. Keluar

Masukkan pilihan: 2

Masukkan ciphertext: gyqmxe

Masukkan ordo matriks key: 2

Masukkan key 2 x 2:

Baris 1: 7 6

Baris 2: 2 5

PLAINTEXT: MAGANG

```
===== HILL CIPHER =====
```

1. Enkripsi
2. Dekripsi
3. Cari Kunci
4. Keluar

Masukkan pilihan: 3

Masukkan ordo matriks key: 2

Masukkan 4 huruf plaintext yang diketahui: help

Masukkan 4 huruf ciphertext pasangannya: hiat

KEY:

3 3

2 5

```
===== HILL CIPHER =====
```

1. Enkripsi
2. Dekripsi
3. Cari Kunci
4. Keluar

Masukkan pilihan: